The effects of the last function we looked at, `mb_detect_order()`, were demonstrated with its complementary function, `mb_detect_encoding()`. By default, `mb_detect_encoding()` uses an internal scoring system to find the best match from the `detect_order` list. It is also possible to force strict matching (system-wide) with the `mbstring.strict_detection` config directive or control the function via an argument.

```
$invalidDoubleByte = "\xC2\x28";
$validUTF8 = "\u{141}ukasz";

print mb_get_info('strict_detection') . PHP_EOL;
var_dump(mb_detect_encoding(
    $validUTF8 . $invalidDoubleByte
));
print mb_get_info('illegal_chars') . PHP_EOL;

var_dump(mb_detect_encoding(
    $validUTF8 . $invalidDoubleByte,
    strict: true
));

Output:
Off
string(5) "UTF-8"
0
bool(false)
```

Here we take the invalid continuation sequence used in previous examples and append it to a perfectly legal UTF-8 string. PHP is configured for non-strict detection, and `mb_detect_encoding()` matches to UTF-8 because despite the invalid *0xC2* byte, it is still closer to UTF-8 than ASCII. Note that the internal `illegal_chars` counter has not been incremented: its use is inconsistent in the `mbstring` source code, and there's no sign of it in the C implementation of the function we're looking at (I checked). Visit https://github.com/php/php-src/blob/php-8.4.0beta4/ext/mbstring/mbstring.c#L3398 for the gory details. On the second call to `mb_detect_encdoing()`, we pass `strict: true`, and this time, the return is `false`, which means the input string was not an exact match for UTF-8. Far safer!

mb_check_encoding()